



AuthN and AuthZ (part 2/2)

Oktober 2026

Recap day 4



Last time: authentication, authorization, roles, and IDOR



We talked about 401 vs. 403 and security testing



The project focus was securing one or two protected routes



Today: modern API security mechanisms and hardening

Today's agenda



Auth mechanisms: basic auth, API keys, bearer tokens



JWT and OAuth/OIDC in practice



Role, scope and resource-based authorization



API hardening, validation, rate limiting, and secrets



Section 1

Auth mechanisms and security trade-offs

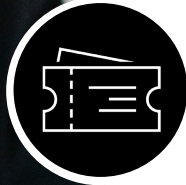
Ways to prove identity



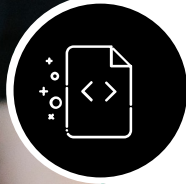
Basic auth



API keys



Bearer tokens



JWT



OAuth/OIDC



Basic auth: simple but limited



Username and password sent in the request



Easy to demo and explain



Requires HTTPS in real use



Not ideal as long-term production pattern

API keys and bearer tokens

- API keys identify a caller or service
- Bearer tokens prove possession of a token
- Tokens are often used with Authorization headers
- Security depends on storage and support



Why JWT matters



JWT is a signed token format



Contains claims such as user, role, and expiry



Often used as stateless API authentication



Claims must be validated, not trusted blindly

OAuth 2.0 and OIDC

- OAuth 2.0 is about delegated authorization
- OIDC adds identity information about the user
- The app gets a token, and the API validates it
- The API still owns the authorization decision



Mini exercise 0

Choose the best mechanism

- Review several use cases
- Match each scenario to the best auth model
- Discuss the trade-offs and risks





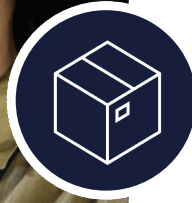
Section 2

JWT, claims, scopes, and tokens

The structure of a JWT



Header



Payload



Signature



The API verifies the signature and claims

Claims and scope

Sub

- User identity

Roles

- Permission groups

Scope

- What access was granted

Exp

- When the token expires



Security pitfalls with tokens



Expired token ignored



Claims not validated

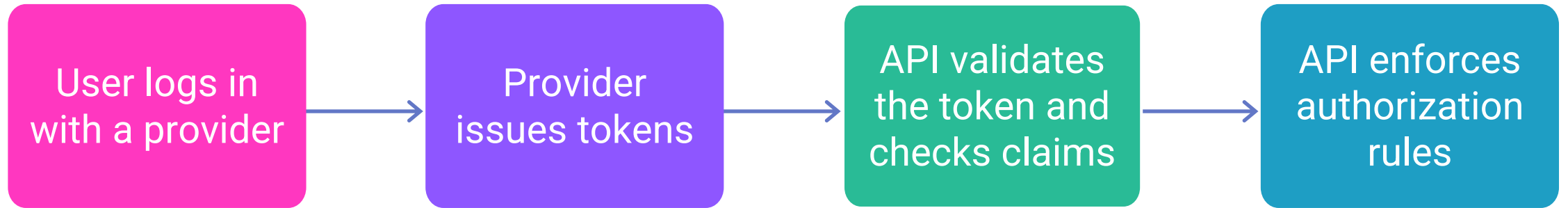


Hardcoded secrets



Refresh token stored insecurely

OAuth/OIDC in one flow



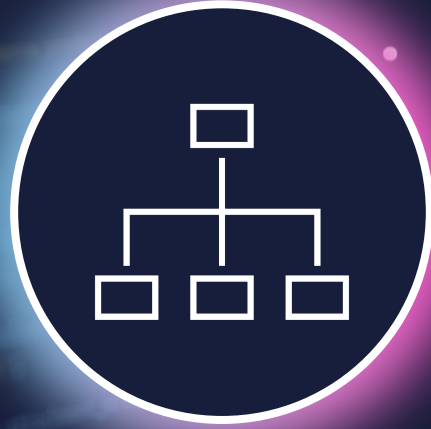
Mini exercise 1

Token review

- Inspect access tokens
- Extend verifier tests to cover payload tampering
- Explain why decoding a JWT does not verify it
- Evaluate the signature and claims checks

Done? Mini exercise 2 is a bonus exercise





Section 3

Authorization models

Role-based access control

- Roles are simple and effective
 - USER
 - ADMIN
 - MANAGER
- Great for common permission groups
- Less precise for resource ownership rules



Resource-based authorization



User can view only their record



User can edit only their own order



The resource context matters



Backend check required

Attribute-based access control

- Authorization based on properties
 - Department
 - Location
 - Time
 - Policy
- Useful for more dynamic rules
- More complex to maintain



Scope and least privilege



Give the minimum access required



Use scopes to limit permissions



Test admin, manager and user flows



Least privilege reduces damage

Mini exercise 3

Implement resource-based authorization

- Use a verified subject
- Prevent broken objev-level authz
- Add tests





Section 4

API hardening

HTTPS is not optional



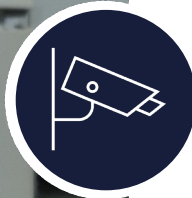
Credentials and tokens travel over the network



HTTPS protects transport security



Without it, auth is exposed



Treat every request as if it can be observed

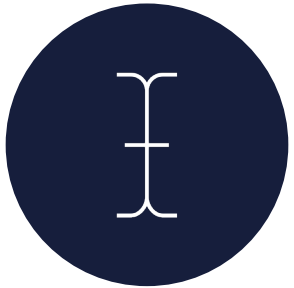
CORS, rate limiting, and validation



CORS controls browser access



Rate limiting protects against abuse



Input validation prevents bad payloads

Secrets management and response hygiene

- Never hardcode credentials or tokens
- Use environment variables or a secrets store
- Do not leak stack traces or internal details
- Keep errors useful, not exposing internals



OWASP API Security Top 10



Broken object level authorization



Broken authentication



Excessive data exposure



Rate limiting and resource abuse



Injection and misconfiguration

Mini exercise 4

Security checklist review

- Review a sample API or project
- Spot the likely security issues
- Fix them and test them





Summary



Auth mechanisms and their trade-offs



JWT, claims, scopes, OAuth/OIDC



RBAC, ABAC, resource checks, and least privilege



Hardening: HTTPS, validation, rate limiting, secrets, and OWASP

Project focus for the afternoon

- Add a token or basic auth flow
- Protect a route with authorization checks
- Write a 401, 403, and IDOR test
- Review the API with a security checklist